

Technical walkthrough

A production-minded Spring Boot take-home: stable artist identity, aliases, tracks, deterministic Artist of the Day, and a deployable foundation.

Java 25 ♦ Spring Boot 4

PostgreSQL ♦ Flyway

CI • Docker • Observability

15-minute presentation aid

Jad El Omeiri

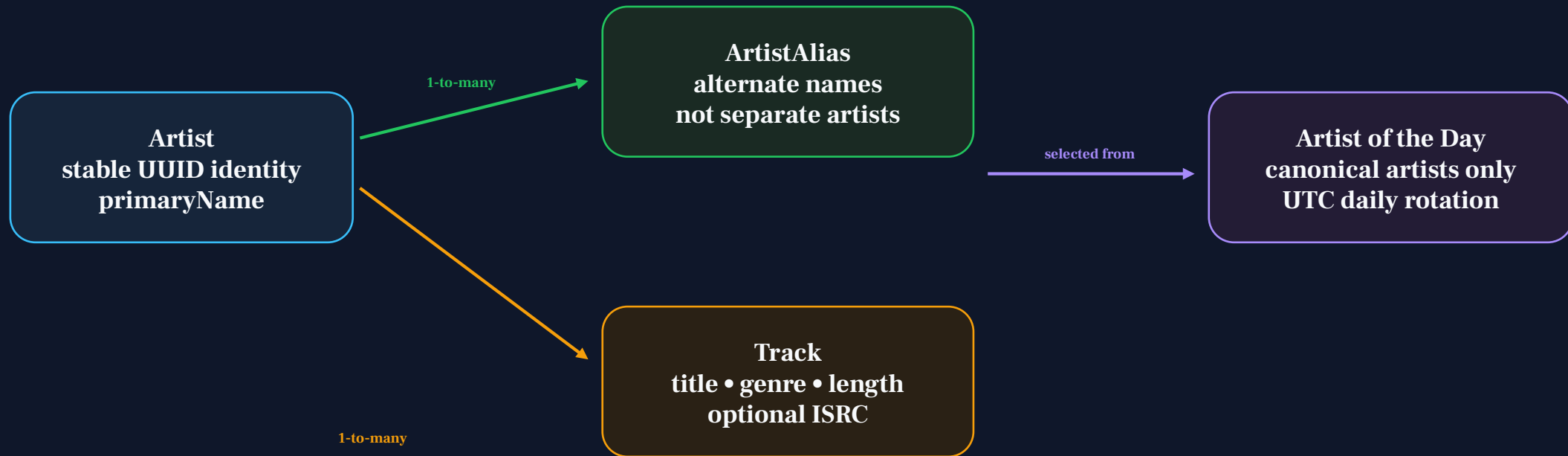
What I optimised for

A music metadata service needs stable identity first — because names, aliases and display metadata are messy.



Key guardrail: build enough production foundation to be credible, but do not turn a take-home into a platform rewrite.

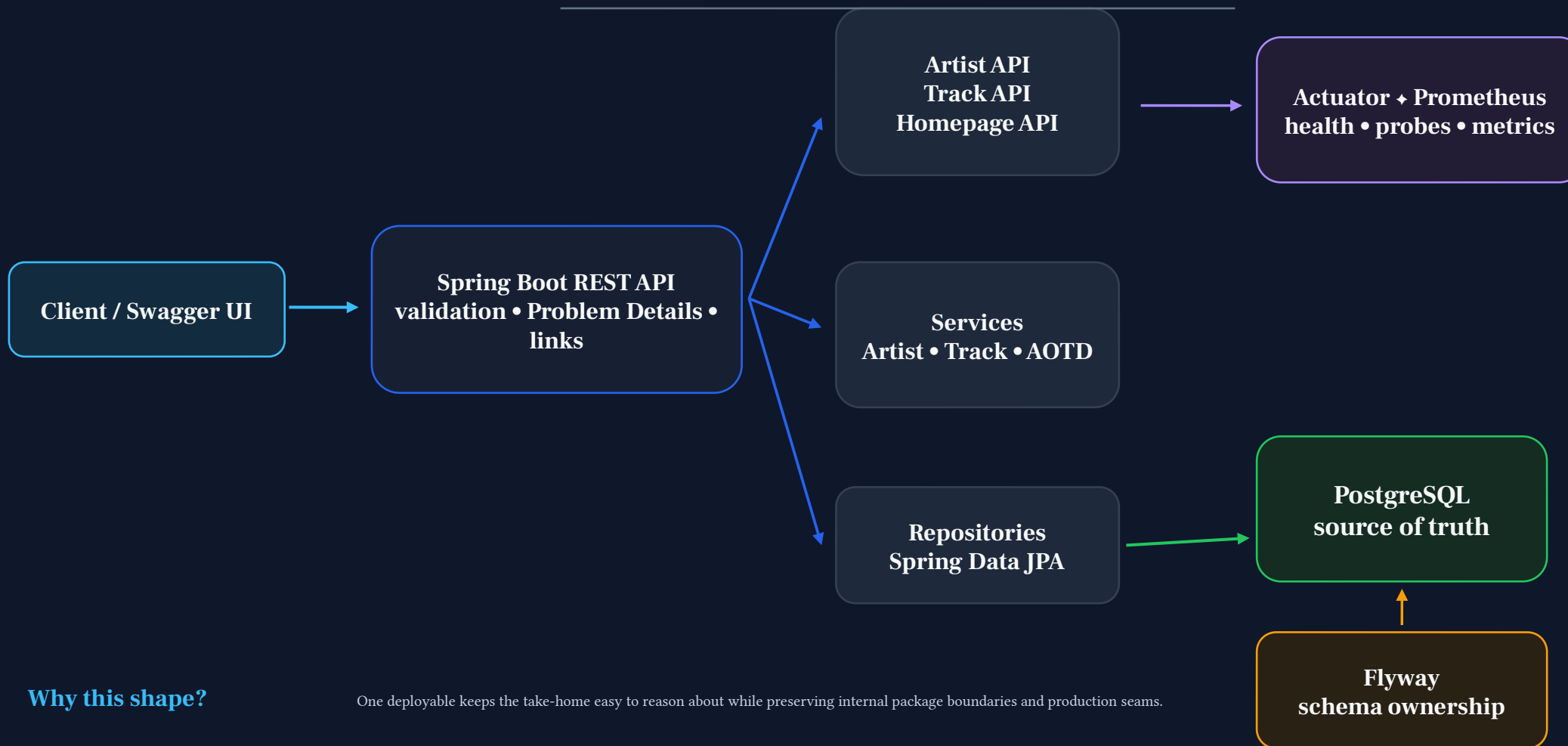
Domain model: stable identity, messy names



Design implications

Aliases are metadata, not identities. Track retrieval is bounded because prolific artists can have large catalogues. Optional genre/ISRC fields are treated pragmatically rather than forcing a taxonomy too early.

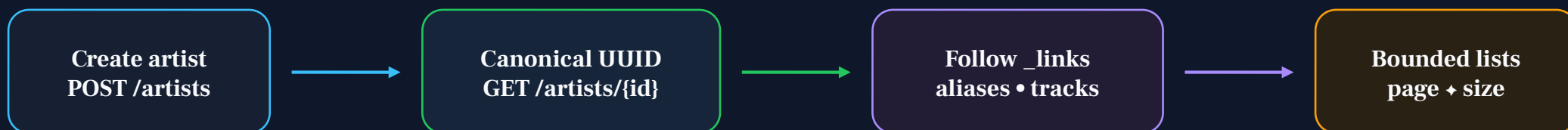
Current architecture: modular monolith with clear seams



Why this shape?

One deployable keeps the take-home easy to reason about while preserving internal package boundaries and production seams.

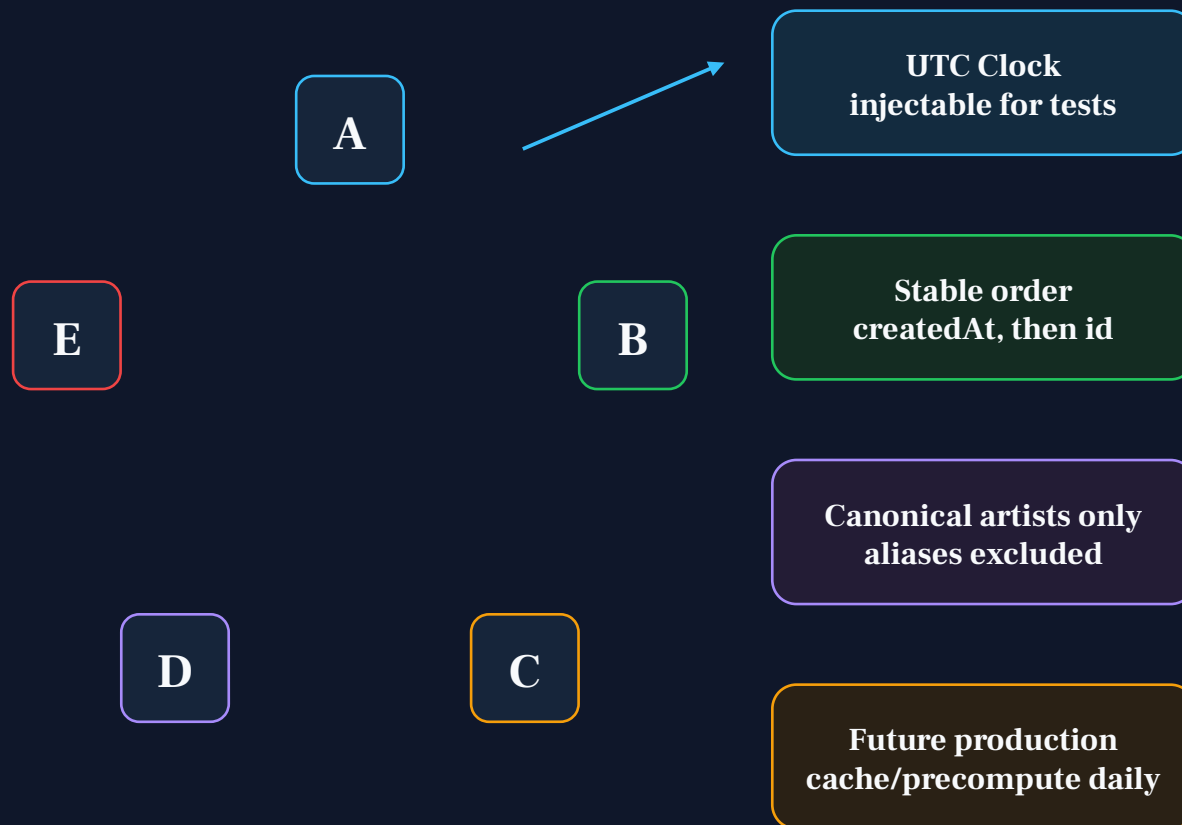
API choices: stable resources, discoverable responses



Trade-offs I chose deliberately

- UUIDs are canonical because artist names, aliases and track titles are not guaranteed unique or stable.
- Lightweight HATEOAS improves exploration without turning the task into a hypermedia exercise.
- Problem Details make failure modes consistent and client-friendly.
- Artist/track discovery and slugs are documented as future API usability improvements, not rushed into this submission.

Artist of the Day: deterministic, fair, testable



$\text{selectedIndex} = \text{daysSinceEpoch} \% \text{artistCount}$

Why not a stored pointer?

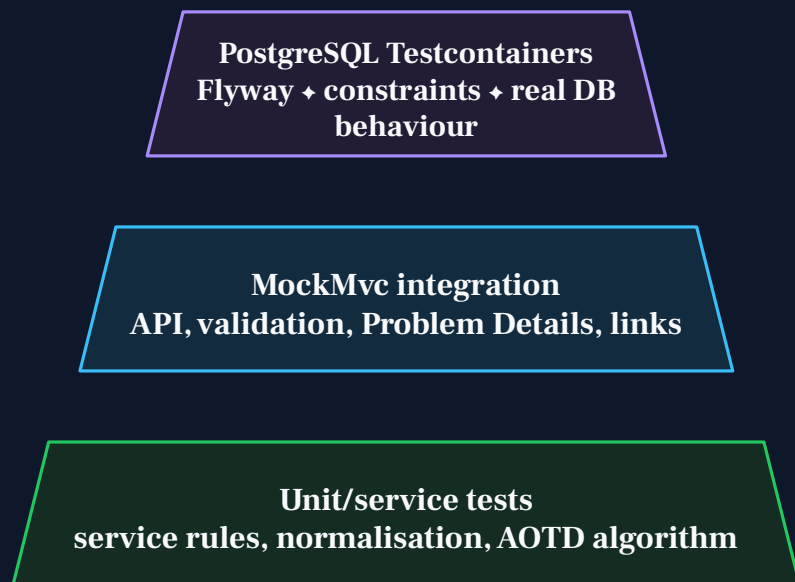
A mutable pointer introduces scheduling, failure and multi-instance coordination questions. The modulo approach derives the result from the date, making the take-home version stateless and repeatable.

Known trade-off

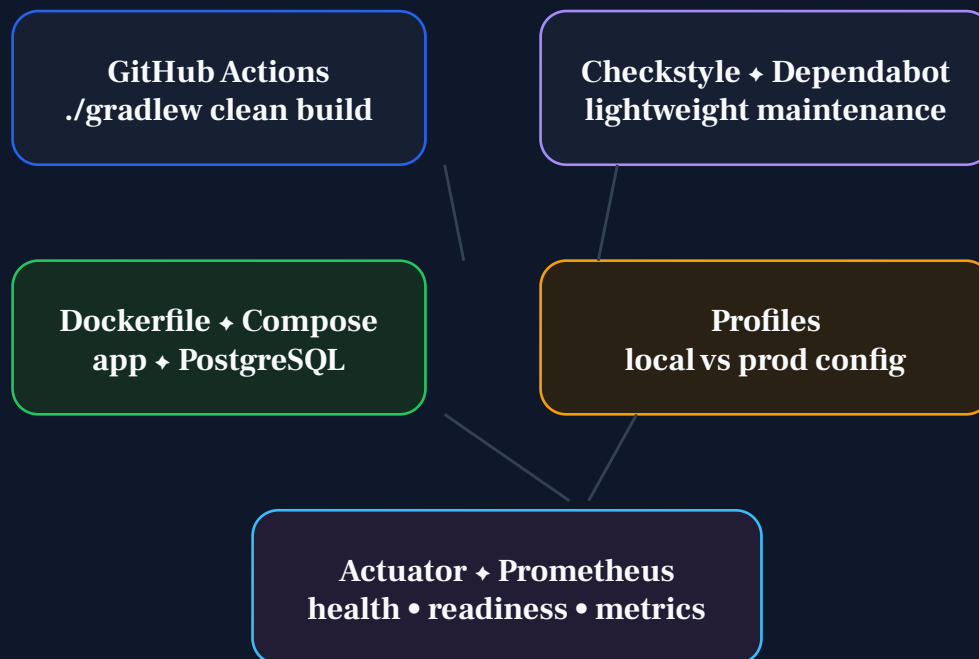
If artists are added during the day, today's selection can shift. In production I would store or cache the daily result until the next UTC day.

Confidence: fast tests ♦ realistic integration ♦ operability

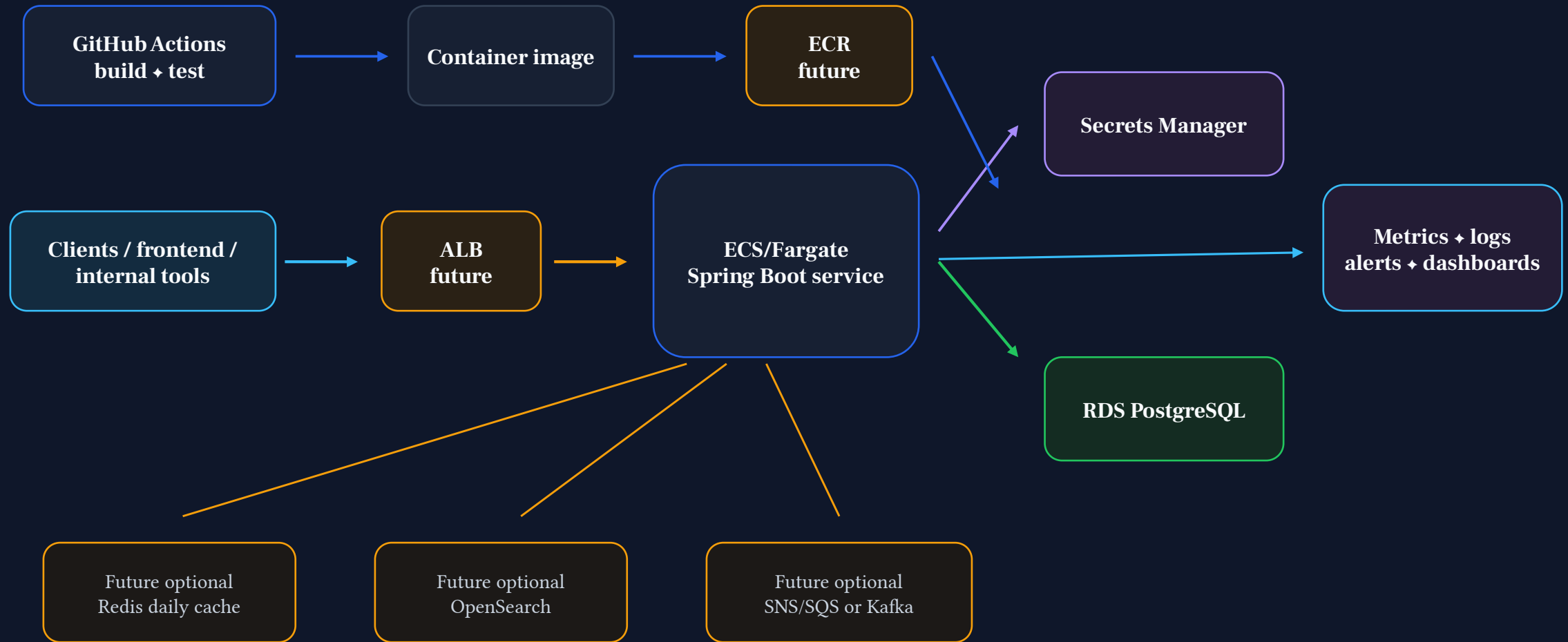
Testing pyramid in this repo



Production-readiness signals



Future production path: evolve only when needed



Not implemented in the take-home — shown as a realistic production evolution path.

How I would present it in 15 minutes



The story I want to land

I understood the product shape, modelled the domain around stable identity, kept the architecture intentionally simple, and added enough production-readiness to make the service reviewable, runnable and testable.

Next steps I would discuss: auth for writes, artist/track discovery endpoints, cached or precomputed Artist of the Day, search infrastructure when needed, and production deployment/observability maturity.

Submitted repo includes README, decision log, production-readiness notes, diagrams and presentation outline.